RV College of Engineering®

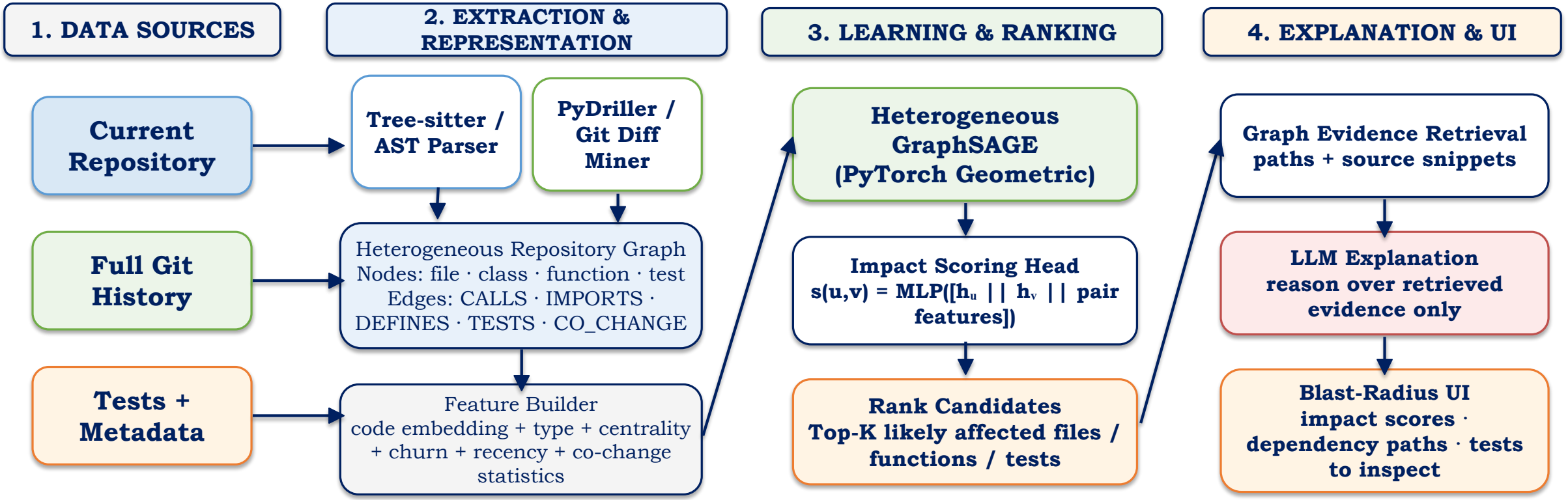


Neural Cartographer: Repository Intelligence and Change-Impact Prediction

RV College of Engineering

System Architecture — End-to-End

The system has four layers: repository ingestion, graph/feature construction, predictive learning, and evidence-backed explanation.



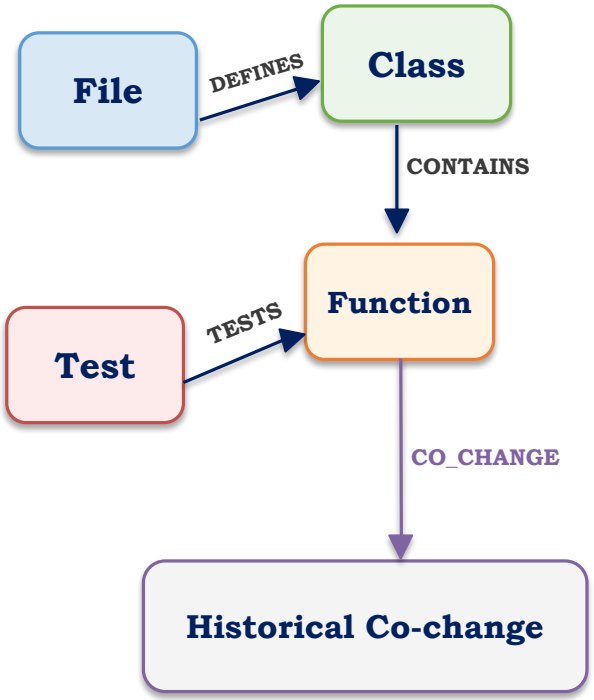
Methodology split: OFFLINE TRAINING uses past commits as supervision; ONLINE INFERENCE freezes the current repository graph, scores candidates for a user-selected seed component, and only then invokes the LLM for explanation.

RV College of Engineering

Architecture Deep Dive — Graph Schema & Model

The ML core is a graph-ranking problem: given changed node u, learn to rank candidate node v by future co-change/impact likelihood.

A. Repository Graph Schema



Other edge types: CALLS, IMPORTS, INHERITS, READS/WRITES, SAME_MODULE

B. Node / Pair Features

Node feature vector x_v
- pretrained code embedding (e.g., UniXcoder / CodeBERT)
- node type + module depth
- degree / PageRank / betweenness

Evolution features
- churn count + recent edit frequency
- last-change recency
- co-change count / Jaccard / PMI
- ownership / author overlap (optional)

Pair features $\phi(u,v)$
- shortest-path distance
- edge-type path summary
- semantic cosine similarity
- historical co-change strength

C. Predictive Model

2-layer Hetero GraphSAGE message passing per relation type

↓

Pair scorer
$s(u,v)=\text{MLP}([h_u \,||\, h_v \,||\, \phi(u,v)])$

↓

Training objective
BCEWithLogits + hard-negative sampling (optional pairwise ranking loss)

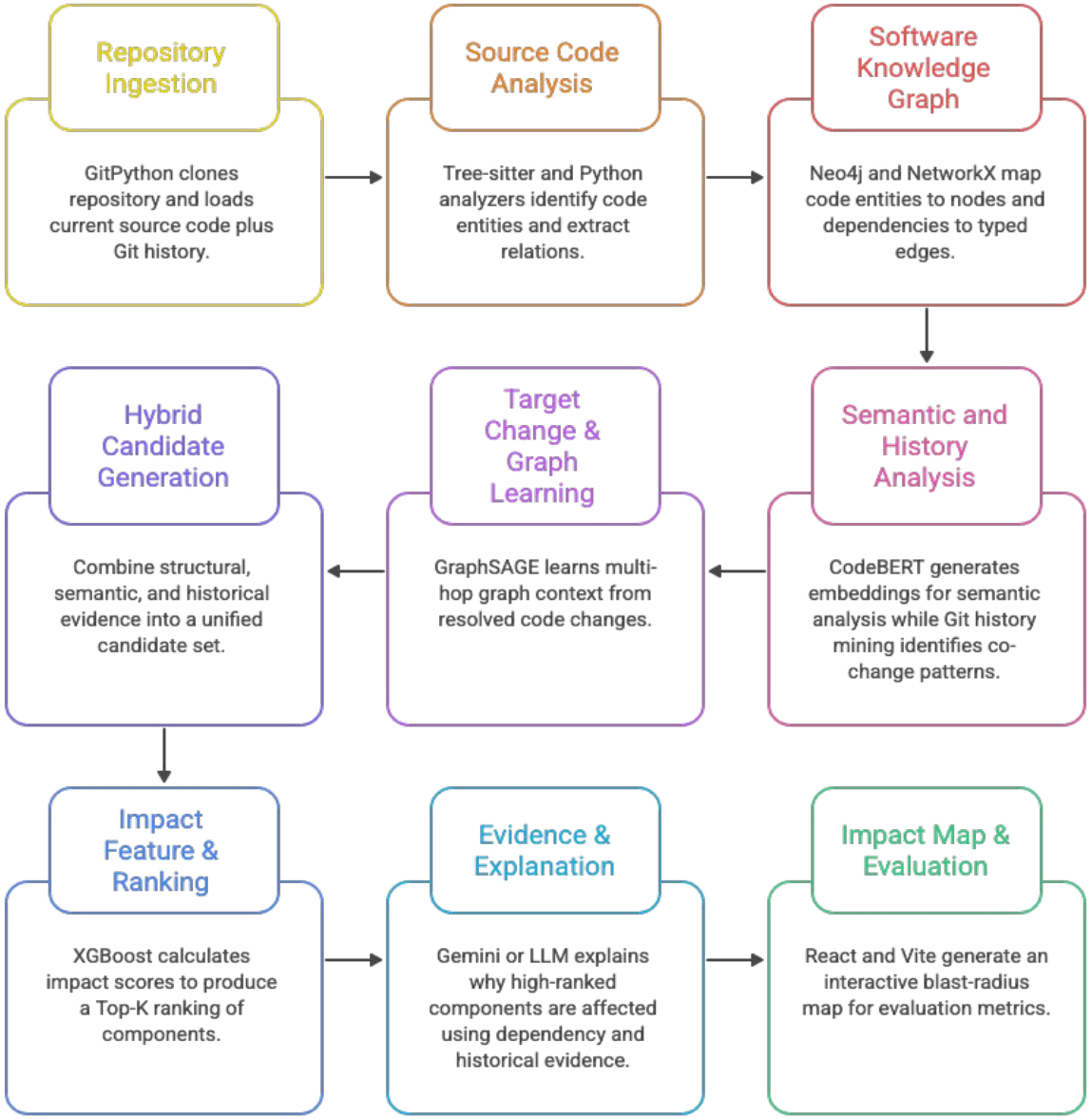
↓

Output: ranked candidates $v_1 \ldots v_{\square}$

Implementation stack: Tree-sitter · PyDriller/GitPython · NetworkX/Neo4j · PyTorch Geometric · FAISS/Qdrant · FastAPI · React/Cytoscape.js

RV College of Engineering

System Architecture



Made with Napkin

RV College of Engineering

Methodology — Offline Training & Online Inference

Training and prediction are deliberately separated so that the LLM never creates impact labels or scores.

OFFLINE — MODEL BUILDING

1	**Clone + normalize repository**	Pin commit history, ignore generated/vendor files; detect Python modules and tests.
2	**Build static program graph**	Parse AST; create file/class/function/test nodes and CALLS/IMPORTS/TESTS edges.
3	**Mine Git evolution**	For each commit, map diffs back to changed files/functions and create temporal co-change edges.
4	**Create supervised samples**	Seed changed node u; positives = other changed nodes; hard negatives = plausible but unchanged neighbors.
5	**Generate embeddings + graph features**	Compute code embeddings, graph statistics, churn/recency and co-change features.
6	**Train + tune GNN**	Train on earlier commits; choose hyperparameters using validation repo / later validation window.

ONLINE — CHANGE IMPACT QUERY

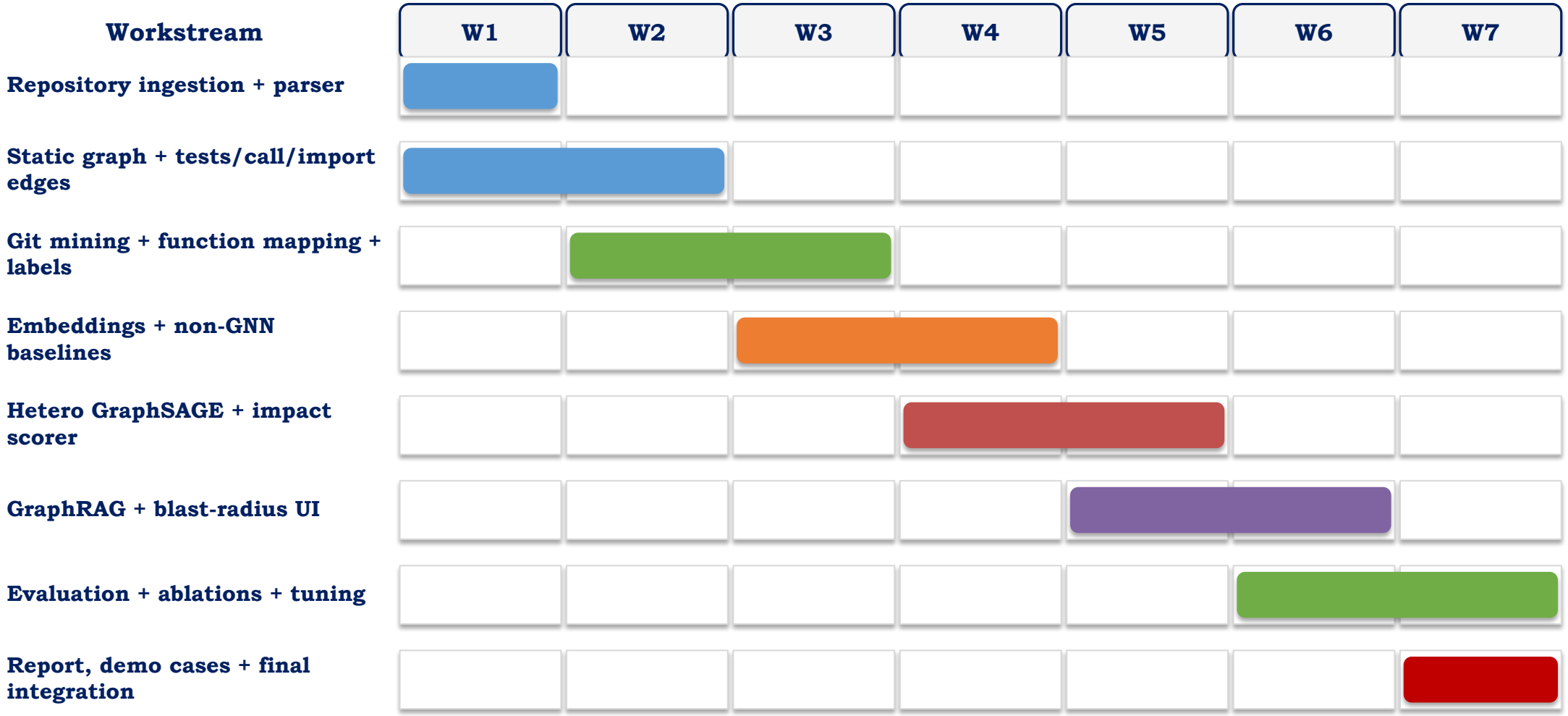
1	**Select seed component**	Developer selects a file/function/class planned for modification.
2	**Freeze current graph**	Use the latest source snapshot; no future Git information is exposed.
3	**Generate candidate set**	2–3 hop structural neighbors + historically related nodes + semantic nearest neighbors.
4	**GNN impact scoring**	Predict P(v affected | u changed) for every candidate and sort descending.
5	**Retrieve supporting evidence**	Fetch shortest dependency paths, co-change history and relevant code snippets.
6	**Explain + visualize**	LLM verbalizes retrieved evidence; UI highlights Top-K nodes and recommended tests.

Key rule: prediction = learned graph model; explanation = retrieved graph evidence + LLM. The LLM is not used as the predictor.

RV College of Engineering

7-Week Implementation Plan

Goal: reach an end-to-end, evaluated prototype by Week 7 — not just a graph visualizer.



Milestone A — end W3: dataset + graph builder stable

Milestone B — end W5: trained predictor beats simple baselines

Milestone C — end W7: evaluated demo + report + reproducible pipeline

Go, change the world

RV College of Engineering

Codebases Used — Repository-wise Split

Python-only first phase keeps AST mapping consistent. Repositories are split by project so no project-specific patterns leak into testing.

TRAINING (5)

- Pallets / Flask — compact framework; fast iteration
- psf / Requests — mature library; clear cross-file behavior
- pytest-dev / pytest — rich test relationships
- sphinx-doc / sphinx — modular plugin/extension architecture
- pydantic / pydantic — evolving modern codebase

VALIDATION (1)

- fastapi / fastapi — unseen but related framework
- Used for model selection / thresholds only

HELD-OUT TEST (3)

- sympy / sympy — large function-heavy system
- scikit-learn / scikit-learn — modular ML library + tests
- django / django — large framework / scalability stress test

Repository selection criteria
• long public Git history • meaningful cross-file structure • substantial tests
• active maintenance • permissive/open-source availability

Data extracted per repository
current source + AST relations + full commit metadata + file/function diffs + test locations + rename mapping

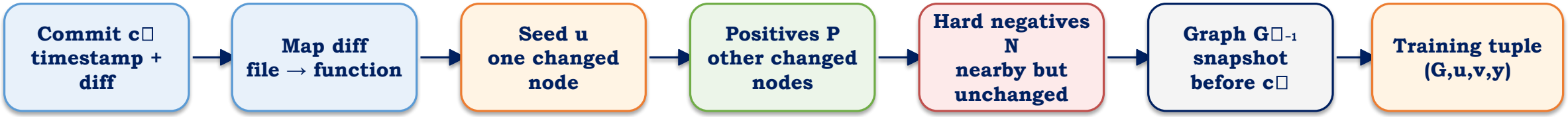
Later extension: Java/TypeScript only after the Python pipeline is stable.

Go, change the world

RV College of Engineering

Dataset Construction — Git History → Supervised Labels

Each historical commit is converted into ranking examples using only information available before that commit.



Example commit
Changed: auth.py::generate_token(), middleware.py::verify_token(), tests/test_auth.py::test_expiry()

Seed: generate_token()
Positive candidates: verify_token(), test_expiry()
Hard negatives: nearby auth functions not changed in this commit

Leakage & noise controls
- chronological snapshots: features use only commits < c□
- project-wise held-out testing
- ignore merges / formatting-only / generated-file changes
- rename tracking before diff-to-function mapping
- file-level fallback if function mapping is ambiguous
- hard negatives sampled from plausible candidate neighborhood

Stored training row / edge sample
repo | commit_time | seed_node | candidate_node | label y | graph_distance | relation_path | code_similarity | cochange_count | recency | churn

Primary label: y=1 when candidate v is part of the same future change set as seed u; y=0 for sampled plausible non-impacted candidates.

RV College of Engineering

Evaluation Metrics — Ranking Quality After Training

Ground truth = components that actually changed with the seed in future commits. Model output = ranked candidate list.

Recall@K	**|TopK ∩ P| / |P|**	Coverage of the true impact set; primary for change-impact analysis.
Precision@K	**|TopK ∩ P| / K**	How many recommended components are actually impacted.
MAP@K	**mean of Average Precision**	Rewards returning all relevant components early in the ranking.
NDCG@K	**DCG@K / IDCG@K**	Position-sensitive ranking quality; strong penalty for burying relevant nodes.
PR-AUC / F1	**thresholded impact scores**	Secondary classification view; PR-AUC is useful under heavy class imbalance.

Primary report: Recall@5, Recall@10, Precision@5, MAP@10, NDCG@10. Secondary: PR-AUC and F1 after fixing a score threshold on validation data.

RV College of Engineering

Evaluation Protocol, Baselines & Success Criteria

The final claim is not "the graph looks useful"; it must outperform simpler retrieval and co-change baselines on unseen repositories.

A. Evaluation Protocol

- Project-wise held-out test: SymPy, scikit-learn, Django never used for training.
- Chronological evaluation: predict later commits from earlier graph/history only.
- Macro-average metrics per repository to avoid one huge repo dominating.
- Report 95% bootstrap confidence intervals across query instances.
- Candidate ranking excludes the seed itself and uses identical candidate pools across models.

B. Baselines / Ablations

- Random / popularity baseline
- Static graph distance only
- Historical co-change frequency only
- Code-embedding cosine similarity only
- Static-only GraphSAGE
- History-only GraphSAGE
- Full fusion: static + semantic + historical

C. Success Criteria

- Full fusion beats the strongest baseline on held-out Recall@5 and NDCG@10.
- Ablation shows that removing static, semantic or history signals degrades ranking quality.
- Explanations cite retrieved dependency/co-change paths rather than unsupported LLM claims.
- Demo cases show Top-K affected tests/functions for a real proposed code change.
- Inference latency is measured separately from LLM explanation latency.

Final experiment: for each future commit, reveal one changed component as the seed and test whether Neural Cartographer ranks the remaining actually changed components near the top.